بيانك
Bayanuk

Technical Plan

Architecture, stack, AI pipeline, API, security, testing, and deployment for the Bayanuk MVP.

Product	Bayanuk (بيانك), AI review of job offers and employment contracts
Owner	JX Tech Studio
Version	1.2 (updated for the API-based AI stack)
Date	October 2026

Changes in v1.2

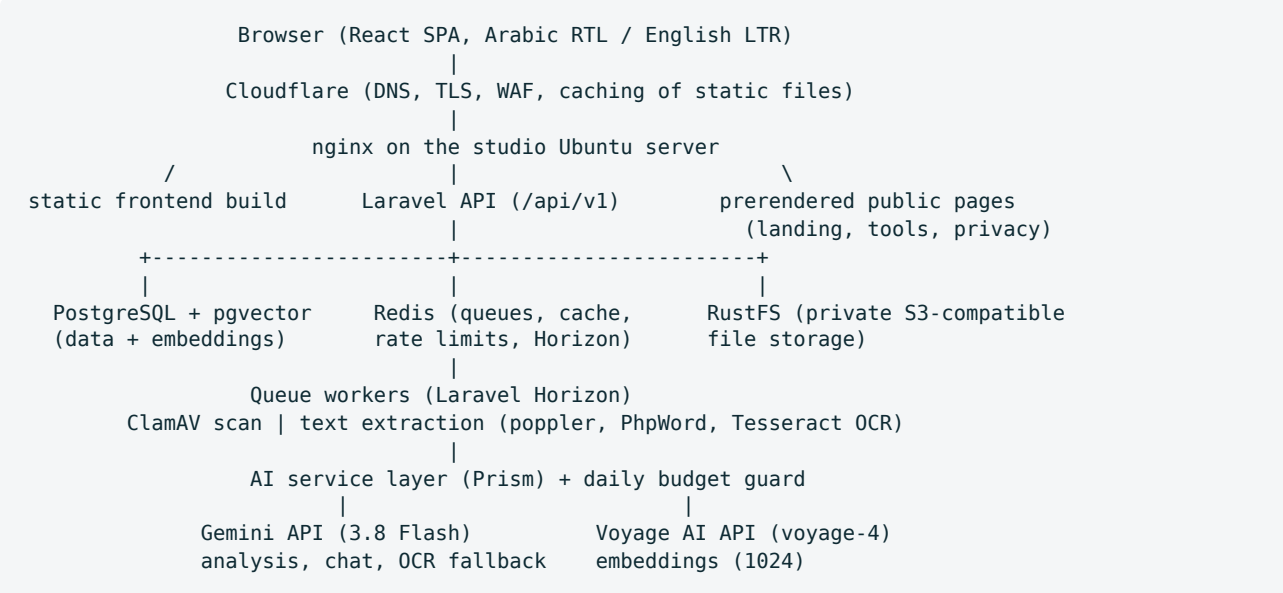
This version applies the AI Stack Decision v2.0: Gemini 3.8 Flash for analysis and chat, voyage-4 for embeddings, and no AI models hosted on our server. It replaces v1.1 (self-hosted models). Only the AI-related parts changed since v1.0; changed rows and lines are marked v1.2. The API section (7) is unchanged and will be updated after the frontend and backend integration contract is approved.

Section	Change
3 Technology stack	Named AI models and providers; OCR falls back to Gemini instead of a separate vision model.
5 Pipeline	Staged analysis; OCR fallback; daily budget check before AI calls.
6 AI design	Providers and models per task; no reranker at the start; privacy on paid plans; $2 daily AI budget.
8 to 12	Secrets, testing, local development, cost monitoring, and risks updated.

1. Purpose

This plan explains how Bayanuk is built: architecture, stack, the document processing pipeline, AI design, API, frontend structure, security, testing, and deployment. It is the technical reference for the backend (Laravel) and frontend (React) work. Product scope lives in the MVP Definition; the data model lives in the Database Schema; model and provider choices live in the AI Stack Decision.

2. Architecture overview



Everything runs on the studio's own Ubuntu server with Docker Compose, the same way the studio already runs its other products. No AI model runs on our server. Only the masked text each task needs is sent to Gemini and Voyage AI; files, results, and embeddings stay on our server. v1.2

3. Technology stack

Layer	Choice	Why
Frontend	React + TypeScript, built with Vite	Team skill; fast builds; strict types catch API mismatches early.
UI	Tailwind CSS (logical properties for RTL) + shadcn/ui (Radix)	Accessible components; Radix supports RTL through a direction provider.
Data fetching	TanStack Query	Caching, polling of processing status, retries.
Routing / i18n	React Router, react-i18next	Arabic and English with dir switching.
Document viewer	react-pdf (pdf.js)	Show the original file and jump to the cited page.
Brand	Readex Pro font; colors from the logo kit	One family for Arabic and Latin.
Backend	Laravel (latest stable), PHP 8.3+	Team skill; mature queues, auth, storage, validation.
Auth	Laravel Sanctum (SPA cookie auth) + Socialite (Google) + custom magic link	No passwords to store or leak.
Queues	Redis + Laravel Horizon	The pipeline is asynchronous; Horizon gives visibility and retries.
AI integration	Prism (prism-php) behind our own AIService interface	One interface for all AI tasks; providers and models are configuration values. v1.2
AI models	Gemini 3.8 Flash (Gemini API, paid tier) for classification, extraction, chat, and OCR fallback; voyage-4 (Voyage AI) for embeddings	High quality in Arabic and English at low cost; no GPU server to run. v1.2
Database	PostgreSQL 16+ with pgvector	Relational data and vector search together.
File storage	RustFS (S3-compatible, self-hosted), Laravel S3 disk	Same as invoi; files never leave our server.
Text extraction	poppler pdftotext (spatie/pdf-to-text), PhpWord for DOCX	Fast and free for text PDFs.
OCR	Tesseract with Arabic + English data; Gemini reads the page image when OCR fails	Scanned documents; the API is used only for pages Tesseract cannot read. v1.2
Malware scan	ClamAV daemon	Every upload is scanned before processing.
CI/CD	Jenkins	Already used by the studio.
Monitoring	Prometheus + Grafana, Laravel logs, Horizon dashboard	Already installed on the studio server.

4. Repositories and structure

Two repositories: `bayanuk-web` (frontend) and `bayanuk-api` (backend), plus an infra folder in the API repository for Docker Compose and nginx files. Branches: main (production), develop (staging), and short-lived feature/* branches merged by pull request after review.

bayanuk-web (feature-based)

```
src/
  app/              router, providers (QueryClient, i18n, direction), layout
  features/
    upload/         UploadDropzone, file checks, upload mutation
    analysis/       ProcessingScreen, ResultDashboard, TermCards, FindingsList, Summary
    chat/           ChatPanel, SuggestedQuestions, CitationChip
    viewer/         DocumentViewer (react-pdf), page jump
    auth/           MagicLinkForm, GoogleButton, guest claim
    documents/      MyDocuments list, delete dialog
    compare/        (MVP+) CompareView
  components/       shared UI (Button, Card, Badge, Dialog...) from shadcn/ui
  lib/              api client (axios/fetch wrapper), formatters (currency, dates, numerals)
  locales/          ar.json, en.json
```

bayanuk-api

```
app/
  Http/Controllers/Api/V1/     DocumentController, ChatController, AuthController, ...
  Http/Requests/               UploadDocumentRequest, AskQuestionRequest, ...
  Http/Resources/              DocumentResource, AnalysisResource, FindingResource
  Models/                      User, Document, DocumentPage, DocumentChunk, Analysis, ...
  Enums/                       DocumentStatus, DocumentType, Severity, ...
  Jobs/Pipeline/               ScanFile, ExtractText, ClassifyDocument, AnalyzeDocument,
                               DetectFindings, IndexChunks, DeleteDocument
  Services/AI/                 AIService (interface), PrismAIService, Prompts/, Schemas/
  Services/Extraction/         PdfTextExtractor, DocxTextExtractor, OcrExtractor, PiiMasker
  Services/Rules/              FindingRules (config-driven), RuleEngine
  Policies/                    DocumentPolicy (owner or guest token)
config/bayanuk.php             limits, retention, rule thresholds, model names, prices,
                               daily AI budget                                    [v1.2]
resources/prompts/             versioned prompt templates (v1/classify.md, v1/extract.md ...)
tests/                         Feature/, Unit/, Golden/ (sample documents + expected JSON)
```

5. Document processing pipeline

Upload returns immediately with status uploaded. The rest runs as a chained queue batch; each job updates documents.status so the frontend can show progress.

Step	Job	What it does	On failure
1	ScanFile	ClamAV scan; verify real MIME type; count pages (max 30).	rejected (unsafe or too large); file deleted.

Step	Job	What it does	On failure
2	ExtractText	pdftotext per page; if a page has almost no text or the Arabic text is broken, OCR it (Tesseract ara+eng) and mark is_scanned. If OCR still fails, send the page image to Gemini (extraction_method = vision). DOCX via PhpWord, split into logical pages. Clean text, normalize Arabic numerals, mask national ID, iqama, IBAN, and phone numbers.	failed: "We could not read this file." v1.2
3	ClassifyDocument	Short Gemini call on the first pages only: job_offer, employment_contract, or other; detect language.	other leads to rejected with a polite message. v1.2
4	AnalyzeDocument	The only call that sends the full contract text, with the extraction JSON schema. Validate output; verify every quote exists in its page text; retry once on invalid output. Save analyses.result and extracted_terms. Generate summary.	failed after retry; error_code stored. v1.2
5	DetectFindings	Run deterministic PHP rules on extracted_terms first, then AI checks for clauses rules cannot detect (non-compete, penalties, one-sided changes), using relevant chunks instead of the full text.	Analysis still shown without findings; flagged in logs. v1.2
6	IndexChunks	Split pages into chunks (about 500 tokens, 15% overlap, never across pages); create voyage-4 embeddings (1024); save to document_chunks.	Chat disabled for this document with a message; rest works. v1.2
7	Mark ready	status = ready; processing_finished_at set; usage_events written (tokens and cost).	v1.2

> Steps 5 and 6 run in parallel after step 4. Each job is idempotent (safe to retry) and has a timeout. Failed jobs are visible in Horizon and alert the team through Grafana. **Every job that calls an AI API checks the daily AI budget first.** When the budget is reached, new analyses pause with a clear message until the next day. v1.2

6. AI design

6.1 Provider abstraction v1.2

Controllers and jobs depend on an AIService interface with four methods: `classify()`, `extract()`, `answer()`, and `embed()`. The implementation uses Prism, so the provider and model for each task are configuration values. In the MVP, `classify()`, `extract()`, `answer()`, and the OCR fallback use Gemini 3.8 Flash, and `embed()` uses voyage-4. No code outside `Services/AI` knows which provider is used, so a provider can be replaced without changing the pipeline. A backup provider is still to be decided.

6.2 Structured output and anti-hallucination checks

- Extraction uses a JSON schema (structured output). The response is validated with Laravel validation rules before saving.
- Every value must come with a page number and a short quote. The backend checks that the quote actually appears in that page's text (after normalization). A value with a quote that cannot be found is downgraded to "not found".
- Totals are recomputed from components; a mismatch becomes a "Salary does not add up" finding instead of being silently corrected.
- The model is told to return "not found" instead of guessing. Prompts include Arabic and English examples.
- Prompts are files in the repository with a version number saved on each analysis, so results can be traced and compared after prompt changes.
- Staged analysis: the full contract is sent once, for extraction. Classification uses the first pages, findings use extracted terms and relevant chunks, and chat uses retrieved chunks only. v1.2
- Low temperature (0 to 0.2) for repeatable results. v1.2

6.3 Chat (retrieval-augmented generation)

1. Embed the question with the same embedding model (voyage-4). v1.2
2. Vector search inside the document's chunks only (top 6), plus a keyword match for numbers and clause words.
3. If the best match is below a similarity threshold, answer "not found in your document" without calling the model.
4. Otherwise send the chunks with page numbers and the question; the model must cite pages and quotes in a fixed JSON format.
5. The backend validates citations against the sent chunks before returning the answer.

No reranking model at the start. Retrieval quality is measured on the golden set first, and a reranker is added only if the tests show it is needed. v1.2

6.4 Privacy in AI calls v1.2

- Personal identifiers are masked before any text is sent (national ID, iqama, IBAN, phone, email). Masking alone does not anonymize the contract owner, since the contract text can still identify them.
- Gemini is used on the paid tier. Under Google's published terms, paid-tier prompts and responses are not used to improve Google's products; the data is still processed by the provider.
- Review each provider's processing, retention, and legal terms, including data transfer outside Saudi Arabia under the PDPL, before sending real contracts. Name the providers in the privacy policy.
- Send only what each task needs (classification sends the first pages only; chat sends only retrieved chunks).
- Development and testing use sample or anonymized contracts only.

6.5 Cost control v1.2

- Token usage and cost are saved on every analysis and chat message.
- Daily AI budget: $2, set in `BAYANUK_DAILY_AI_BUDGET_USD` and checked by the application before every AI call. Provider spending alerts are not used as a hard stop. Reviewed after the first 50 real analyses. v1.2
- Limits: guests 2 uploads per day per device and IP; users 10 per day in MVP; chat limits per document.
- Re-uploading the same file (same sha256) by the same owner reuses the existing analysis.
- Limited retries; OCR fallback to Gemini only for pages Tesseract cannot read. v1.2

7. API (v1)

JSON over HTTPS under /api/v1. Signed-in users use Sanctum SPA cookie sessions. Guests send an X-Guest-Token header (random value created by the frontend and kept in the browser). Errors use one format: `{ "error": { "code": "...", "message": "..." } }` with a translated message.

Method	Endpoint	Description
POST	`/documents`	Multipart upload. Returns 202 with { id, status }.
GET	`/documents/{id}/status`	Lightweight status for polling every 2 seconds.
GET	`/documents/{id}`	Full result: document, latest analysis, terms, findings, summary.
GET	`/documents/{id}/file`	Returns a signed URL (5 minutes) to view the original file.
DELETE	`/documents/{id}`	Deletes everything immediately. Returns 204.
GET	`/documents`	Signed-in user's documents (paginated).
GET	`/documents/{id}/chat`	Chat history and remaining questions.
POST	`/documents/{id}/chat`	Ask a question. Returns answer, citations, answer_found.
POST	`/analyses/{id}/feedback`	Thumbs up/down, optional field and comment.
POST	`/comparisons`	(MVP+) { document_a_id, document_b_id }.
GET	`/comparisons/{id}`	(MVP+) Comparison result.
POST	`/auth/magic-link`	Send a sign-in link to an email (rate limited).
POST	`/auth/magic-link/verify`	Exchange token for a session; claims guest documents.
GET	`/auth/google/redirect`, `/auth/google/callback`	Google sign-in.

Method	Endpoint	Description
GET / POST	`/me`, `/auth/logout`	Current user; sign out.
PATCH / DELETE	`/me`	Update locale or keep_documents; delete account.

Example response: `GET /documents/{id}`

```
{
  "id": "0193...", "status": "ready", "document_type": "employment_contract",
  "language": "ar", "page_count": 12, "expires_at": "2026-11-20T10:00:00Z",
  "analysis": {
    "id": "0193...", "summary": "...", "summary_language": "en",
    "terms": [
      { "key": "compensation.basic_salary", "value": 8000, "unit": "SAR",
        "found": true, "page": 3, "quote": "..." },
      { "key": "notice.employer_days", "found": false }
    ],
    "findings": [
      { "category": "non_compete", "severity": "attention", "title": "Non-compete clause",
        "explanation": "...", "page": 9, "quote": "..." }
    ]
  }
}
```

8. Security

Risk	Control
Malicious files	Type checked by content, size and page limits, ClamAV scan, files never executed or served from the API domain directly.
Access to other people's documents	UUID IDs plus DocumentPolicy on every route (owner user or matching guest token hash). Tests for every endpoint.
File exposure	Private bucket; signed URLs valid 5 minutes; storage keys are random.
Data at rest	Encrypted disks on the server; RustFS server-side encryption; database backups encrypted.
Data in transit	HTTPS only (Cloudflare + nginx), HSTS, secure and SameSite cookies.
Abuse and cost attacks	Rate limits per IP, guest token, and user; daily upload limits; daily AI budget; Cloudflare WAF and bot protection on upload. v1.2
Prompt injection inside documents	Document text is passed as data inside clear delimiters; the model cannot call tools; outputs are schema-validated and quotes verified.
Secrets	API keys and other secrets in server environment variables only; never in the repository; separate keys for staging and production. v1.2
Personal data	PII masking before AI calls; only needed text sent; paid plans; retention and deletion jobs; no document content in logs. v1.2
Dependencies	composer audit and npm audit in the Jenkins pipeline.

9. Testing strategy

- Backend: Pest feature tests for every endpoint (including access by the wrong owner), unit tests for rules, PII masking, numeral normalization, and quote verification. AI calls are faked in tests.
- Golden set: 30 sample documents (15 Arabic, 15 English; offers and contracts; including 5 scanned) written or anonymized by the team, each with an expected JSON. A command (`php artisan bayanuk:eval`) runs the real pipeline and reports accuracy per field. Run before every prompt or model change. v1.2
- Before launch: retrieval quality (decides if a reranker is needed), time to result, chat answer time, scanned and large files, user isolation, and cost per analysis, as listed in the AI Stack Decision. v1.2
- Frontend: Vitest + Testing Library for components (term cards, findings, chat citations); checks in both RTL and LTR.
- End-to-end: Playwright smoke test on staging: upload, wait, see result, ask a question, delete.
- Manual: mobile check on real phones in Arabic and English before each release.

10. Environments and deployment

Environment	Where	Purpose
Local	Docker Compose on each laptop (Laravel Sail-style)	Development with fake AI, or real APIs with sample contracts only. v1.2
Staging	Studio server, separate containers and database	Auto-deployed from develop; team testing and demos.
Production	Studio server (later: dedicated server if load grows)	Deployed from main after approval.

Jenkins pipeline

1. Install dependencies; run linters (Pint, ESLint) and type checks (PHPStan level 6, tsc).
2. Run tests (Pest, Vitest); run composer audit and npm audit.
3. Build Docker images; tag with commit SHA.
4. Deploy to staging automatically (develop) or to production on manual approval (main).
5. Run migrations, restart Horizon workers, run the Playwright smoke test.
6. On failure, roll back to the previous image tag.

Docker Compose services

api (PHP-FPM), horizon (queue workers, 2 to 4 processes), scheduler (retention job), postgres (with pgvector), redis, rustfs, clamav, behind the host nginx. No AI model containers. v1.2 Daily database backups to a separate disk, kept 7 days.

11. Observability

Signal	Where	Alert when
Pipeline failures	Horizon + Prometheus counter	More than 5% failed in 1 hour.
Queue wait time	Horizon metrics	Waiting over 2 minutes.
Time to result	processing_finished_at minus created_at	90th percentile over 90 seconds.
AI cost	usage_events, daily Grafana panel	Daily cost reaches 80% of the $2 budget. v1.2
AI provider errors	Laravel logs and Horizon failed jobs	Error rate from Gemini or Voyage AI rises. v1.2
Errors	Laravel logs (no document content)	Any 5xx spike.
Server health	Prometheus node exporter	Disk over 80%, memory pressure.

12. Technical risks

Risk	Impact	Mitigation
Arabic PDF text comes out reversed or broken	Wrong extraction	Detect garbled text (ratio checks); fall back to OCR, then to Gemini for that page; include such files in the golden set. v1.2
PHP is weaker than Python for document processing	Slower progress	Use proven command-line tools (poppler, Tesseract) called from Laravel; if needed, add a small Python extraction service later without changing the API.
AI returns wrong numbers	Loss of trust	Quotes verified, totals recomputed, every value links to its page, user feedback per field.
AI provider outage or price change	Downtime or cost	Provider abstraction; queue retries; backup provider still to be decided. v1.2
Gemini price doubles in January 2027	Cost	About $0.042 per analysis; the $2 budget covers about half as many analyses. v1.2
Cost abuse through uploads	Unexpected bills	Limits, daily AI budget, Cloudflare protection. v1.2
Contract data processed outside Saudi Arabia	Legal	Review provider terms and PDPL before real contracts; state providers in the privacy policy. v1.2
Single server	Downtime	Backups, documented restore steps tested once before launch.